Overview **0**
Use / Run **2**
Server **3**
1. Read and check settings 3
2. Check collection requests 3
3. Provide the API routes 4
4. Start the chosen process 4
Running the app (Using Server) **5**
drouot.js: get and understand Drouot pages 6
collection.js: run the job 6
Druot.js **7**
1. Downloading pages **8**
2. Helpers for reading HTML **8**
3. The four parsers **8**
How it connects to collection **9**
Collection **10**
What each function does 10
How it resumes 11

Overview

Here is the app's sequence, from start to finish:

1. **Start the server** with `npm start`. It loads `.env`, asks whether you are using a VPN, then asks for the lot limit. Enter `999` for no lot-count limit. The VPN answer is a question only; the app does not check your connection.
2. **The server connects to PostgreSQL**, refreshes the two `collected-*.csv` files from data already saved, and starts the API and background worker. Starting the server alone does **not** create a new collection.
3. **Start a collection** with `POST /api/collections` and your API key. You request a maximum number of auctions; the default is **2**. The API returns a job ID and `queued` immediately.
4. **The worker picks up the job** and downloads Drouot's upcoming-auctions page. It reads auction links from that page and follows listing pages when needed.
5. **It skips auction IDs already stored by this collector.** It keeps looking until it finds the requested number of new auctions or runs out of available listing pages. The older `auctions.csv` is separate and is not checked.
6. **For each selected auction**, it downloads the auction page, reads its details and catalogue-page links, then reads the lot links on those pages. It downloads lot details up to the lot limit you entered. [Collection workflow (line 19)](/Users/dcfinlay/Development/AI/EventsMonitor/src/collection.js:19)
7. **It saves auctions, lot details, and progress in PostgreSQL.** If a job is interrupted, it can resume its selected work without repeating parts marked complete.

8. **When the job finishes**, it refreshes `collected-auctions.csv` and `collected-lots.csv` and sets the job status to `completed`, `partial`, or `failed`. The CSVs contain **all saved database rows**, including rows from earlier jobs.

Check a job with `GET /api/collections/JOB_ID`. There is currently **no button or screen** for starting a collection, and no input for choosing a specific auction URL.

Use / Run

npm start

curl -X POST http://localhost:3000/api/collections \

-H "Authorization: Bearer 3ed885b0ecf9f8d8f468360e3f242ea41493cf1cd069f83a6db4298a47171730" \

-H "Content-Type: application/json" \

-d '{"maxAuctions":2}'

Dont forget curl -X before to run from command line

Action	Endpoint or command
Start the **server**	`npm start` — there is no HTTP endpoint for this
Start a **new collection**	`POST http://localhost:3000/api/collections`
Run the collection	No separate endpoint; the background worker picks up the queued job automatically
Check a job's status	`GET http://localhost:3000/api/collections/2` — replace `2` with its job ID
Resume a failed or partial job	`POST http://localhost:3000/api/collections/2/resume`
Check the server	`GET http://localhost:3000/api/health`

They are **two files in one app**, not two programs you have to start separately. When you run `npm start`, [server.js (line 164)](/Users/dcfinlay/Development/AI/EventsMonitor/src/server.js:164) starts the API and worker. The worker uses both files.

Server

- Starts the app and connects to PostgreSQL.
- Provides the API that accepts requests and returns saved auctions and lots.
- When you request a collection, saves it as a job in PostgreSQL.
- Starts a background worker that downloads the auction and lot information.
- Creates CSV files from the saved information.
- Stops the server and worker when you shut down the app.

[server.js (line 1)](/Users/dcfinlay/Development/AI/EventsMonitor/src/server.js:1) is the app's **entry point and HTTP API**. It reads settings, connects the other modules, accepts requests, and starts the background collection worker. It does not parse Drouot pages itself.

1. Read and check settings

[`settings()` (line 20)](/Users/dcfinlay/Development/AI/EventsMonitor/src/server.js:20)

reads environment variables: the PostgreSQL connection, API key, port, worker switch, HTTP limits, and maximum auction and catalogue page counts.

The `integer()` helper rejects numbers outside the allowed ranges.
[`requireConfiguration()` (line 35)](/Users/dcfinlay/Development/AI/EventsMonitor/src/server.js:35)

requires a database URL and an API key of at least 24 characters when starting the API.

The `.env` file is loaded by the Node command in [`package.json`](/Users/dcfinlay/Development/AI/EventsMonitor/package.json), not by code inside `server.js`.

2. Check collection requests

[`validateCollection()` (line 53)](/Users/dcfinlay/Development/AI/EventsMonitor/src/server.js:53) checks the JSON sent when starting a job. Its defaults are **2 auctions**, **include lots**, and **French and English**. It accepts only Drouot, requires a valid auction count, and checks the language list and `includeLots` value.

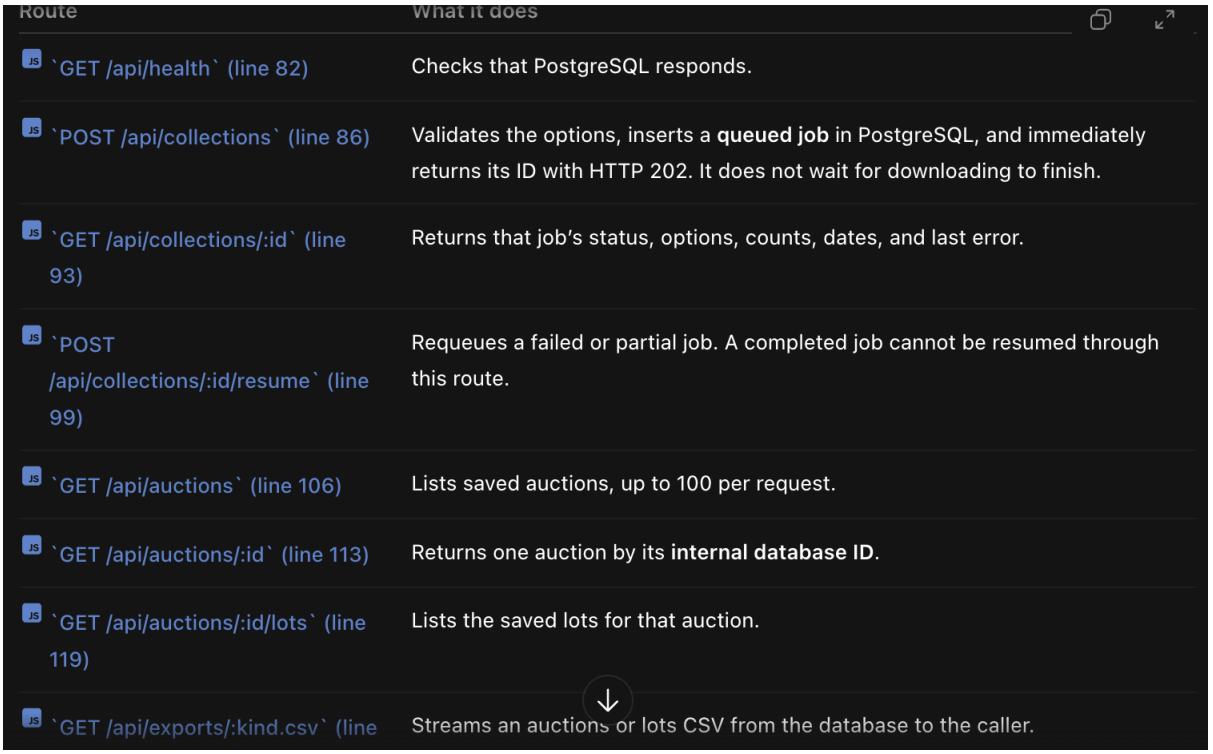
[`validApiKey()` (line 42)](/Users/dcfinlay/Development/AI/EventsMonitor/src/server.js:42) compares the supplied bearer token with the configured API key. The [`auth` middleware (line 75)](/Users/dcfinlay/Development/AI/EventsMonitor/src/server.js:75) applies it to the two operations that change jobs: **start** and **resume**. The read routes do not use that middleware.

3. Provide the API routes

[`createApp()` (line 70)]

(/Users/dcfinlay/Development/AI/EventsMonitor/src/server.js:70) builds the Express app.

It accepts JSON requests up to 16 KB and defines these routes:e list routes use `after` and `limit`: take the last returned row's internal `id` and pass it as `after` to get the next page. Invalid IDs and pagination values receive HTTP 400. Missing records receive HTTP 404. The final [error handler (line 133)](/Users/dcfinlay/Development/AI/EventsMonitor/src/server.js:133) logs unexpected



Route	What it does
`GET /api/health` (line 82)	Checks that PostgreSQL responds.
`POST /api/collections` (line 86)	Validates the options, inserts a **queued job** in PostgreSQL, and immediately returns its ID with HTTP 202. It does not wait for downloading to finish.
`GET /api/collections/:id` (line 93)	Returns that job's status, options, counts, dates, and last error.
`POST /api/collections/:id/resume` (line 99)	Requeues a failed or partial job. A completed job cannot be resumed through this route.
`GET /api/auctions` (line 106)	Lists saved auctions, up to 100 per request.
`GET /api/auctions/:id` (line 113)	Returns one auction by its **internal database ID**.
`GET /api/auctions/:id/lots` (line 119)	Lists the saved lots for that auction.
`GET /api/exports/:kind.csv` (line	Streams an auctions or lots CSV from the database to the caller.

errors and returns a generic error response.

4. Start the chosen process

[`main()` (line 142)](/Users/dcfinlay/Development/AI/EventsMonitor/src/server.js:142) accepts three modes:

- `migrate`: runs the SQL migration, then exits.
- `worker`: runs only the background job loop, with no HTTP server.

- **`api`** (the default): refreshes `collected-auctions.csv` and `collected-lots.csv` from existing database rows, starts the HTTP server, and normally starts the worker in the same process.

In API mode, setting `WORKER_ENABLED=false` leaves collection jobs queued until a separate worker process is started. On Ctrl-C or SIGTERM, `main()` signals the worker to stop, closes the HTTP server, and closes the PostgreSQL pool.

How the files connect: `server.js` puts a job in PostgreSQL through [database.js (line 34)](/Users/dcfinlay/Development/AI/EventsMonitor/src/database.js:34). The worker in [collection.js (line 172)](/Users/dcfinlay/Development/AI/EventsMonitor/src/collection.js:172) claims it and uses [`drouot.js`](/Users/dcfinlay/Development/AI/EventsMonitor/src/drouot.js) to fetch and parse pages. [csv.js (line 31)](/Users/dcfinlay/Development/AI/EventsMonitor/src/csv.js:31) produces the downloadable CSV response and the automatic CSV files.

Running the app (Using Server)

Open a terminal and run:

1. **Start the app:**

```
None

cd /Users/dcfinlay/Development/AI/EventsMonitor

   2. npm start
```

3. Leave this terminal open. You should see `Collector API listening on 3000`.
4. **In a second terminal, start a collection:**

```
None

curl -X POST http://localhost:3000/api/collections \

  -H "Authorization: Bearer YOUR_API_KEY" \

  -H "Content-Type: application/json" \

   5.   -d '{"maxAuctions":2}'
```

6. Replace `YOUR_API_KEY` with the value of `COLLECTION_API_KEY` in your `.env` file. The response gives you a job ID.
7. **Check its progress** (replace `1` with your job ID):

```
None
```

8. `curl http://localhost:3000/api/collections/1`
9. **When it finishes**, look in `/Users/dcfinlay/Development/AI/EventsMonitor` for `collected-auctions.csv` and `collected-lots.csv`.

To stop the app, press **Ctrl-C** in the first terminal.

`drouot.js`: get and understand Drouot pages

[drouot.js (line 1)](/Users/dcfinlay/Development/AI/EventsMonitor/src/drouot.js:1) handles the website itself. It:

- Downloads pages, with delays, timeouts, size limits and retries.
- Rejects URLs outside the approved Drouot hosts.
- Reads the HTML and JSON-LD to find auction links, catalogue pages, lot links, titles and descriptions.
- Returns ordinary JavaScript data such as an auction ID and its French and English URLs.

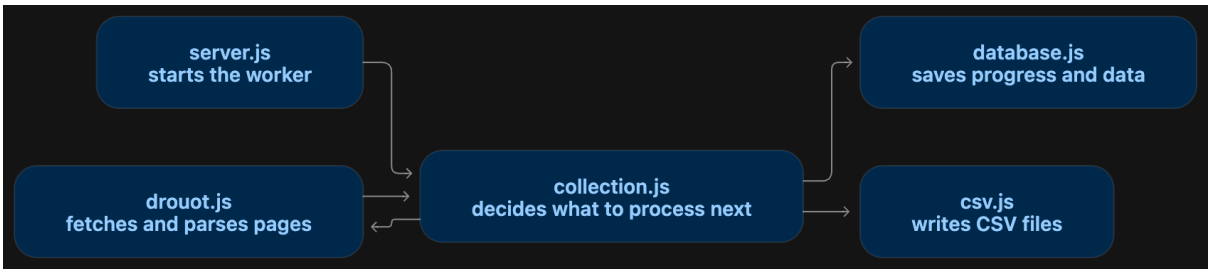
It **does not** decide which job to run, save records to PostgreSQL or create CSVs.

`collection.js`: run the job

[collection.js (line 1)](/Users/dcfinlay/Development/AI/EventsMonitor/src/collection.js:1) controls the sequence of work. It:

1. Claims a queued collection job.
2. Asks `drouot.js` to download and parse the upcoming-auctions listing.
3. Saves each discovered auction through `database.js`.
4. Asks `drouot.js` to fetch and parse auction, catalogue and lot pages.
5. Saves pages and lot details as it goes, recording failures and progress so a job can resume.
6. Calls `csv.js` to create both CSV files when the run finishes.

The direct link is the import at the [top of `collection.js` (line 2)](/Users/dcfinlay/Development/AI/EventsMonitor/src/collection.js:2): it calls the fetching and parsing functions exported by `drouot.js`.



The reason for the split is practical: if Drouot changes its page layout, the parsing changes in `drouot.js`; if you change the order of collection or recovery behavior, that changes in `collection.js`. **You still run only `npm start`.**

Druot.js

Here is how the **current app** collects lots:

1. It opens Drouot's **upcoming-auctions page** and takes the requested number of auction links. You cannot currently give it one specific auction URL.
2. For each auction, it downloads the auction page. Cheerio reads the lot links on that page, such as `/fr/l/12345-item-name`. **It does not guess lot IDs or lot URLs.** [Lot link code (line 202)](/Users/dcfinlay/Development/AI/EventsMonitor/src/drouot.js:202)
3. It looks for catalogue paging links containing `page=`, then requests page 2, page 3, and so on by adding `?page=2`, `?page=3` to the auction URL. It reads the lot links on each page. [Paging code (line 80)](/Users/dcfinlay/Development/AI/EventsMonitor/src/collection.js:80)
4. It visits each saved lot URL, extracts the title and description, and saves the details in PostgreSQL. [Lot collection code (line 49)](/Users/dcfinlay/Development/AI/EventsMonitor/src/collection.js:49)

So the path is: **auction page → catalogue pages → actual lot links → individual lot pages → database**.

[drouot.js](/Users/dcfinlay/Development/AI/EventsMonitor/src/drouot.js) is the part of the app that **talks to Drouot and interprets its pages**. It does not run on its own. `collection.js` calls its functions and saves the returned data.

1. Downloading pages

- [`safeDrouotUrl()` (line 7)](/Users/dcfinlay/Development/AI/EventsMonitor/src/drouot.js:7) checks that a URL uses HTTPS and belongs to `drouot.com` or `www.drouot.com`. It also rejects URLs containing credentials or a custom port. This prevents a link found in a page from sending the collector to another host.
- [`createHttpClient()` (line 16)](/Users/dcfinlay/Development/AI/EventsMonitor/src/drouot.js:16) creates a `get(url, language)` function. It spaces out request starts, sends the appropriate French or English language header, checks redirects, enforces a timeout and maximum response size, and retries transient failures. It also counts requests and retries.

For example, `get(auctionUrl, 'en')` returns the English page's **HTML text**. It does not parse or save that text yet.

2. Helpers for reading HTML

The parser uses Cheerio to select elements from that HTML.

- [`recordsFromJsonLd()` (line 116)](/Users/dcfinlay/Development/AI/EventsMonitor/src/drouot.js:116) reads structured JSON-LD scripts. It can find records inside an `@graph` and skips a malformed script if another one is usable.
- [`sourceLink()` (line 137)](/Users/dcfinlay/Development/AI/EventsMonitor/src/drouot.js:137) extracts a numeric auction or lot ID from a Drouot link. It removes tracking query parameters, keeps the slug, and constructs a canonical French or English URL. This also handles links on the French listing that unexpectedly use an `/es/` prefix.
- [`pageLinks()` (line 149)](/Users/dcfinlay/Development/AI/EventsMonitor/src/drouot.js:149) finds valid `?page=2`, `?page=3`, and similar links for the page being parsed.
- The two text helpers treat fields differently: titles have repeated whitespace collapsed, while descriptions retain their internal line breaks.

3. The four parsers

Function	Input	What it returns
`parseAuctionListing()` (line 162)	Upcoming-auctions listing HTML	Unique auction IDs, French and English URLs, visible page numbers, and an invalid-link count
`parseAuctionPage()` (line 179)	One French or English auction page	Title, description, date and house where available; also catalogue page numbers and lot links
`parseCataloguePage()` (line 215)	One catalogue page	Unique lot links and further page numbers
`parseLotPage()` (line 220)	One French or English lot page	Lot number, title and description; where available, category, currency and image URL

The auction and lot parsers prefer relevant JSON-LD fields and use visible HTML as a fallback. For a lot, the `h1` supplies a title such as `1 - Silver tastevin`; the parser separates `1` from `Silver tastevin`. It uses the description paragraph or Product JSON-LD for the description. If Product JSON-LD identifies a different auction, it reports an error.

How it connects to collection

`collection.js` calls the HTTP client to obtain HTML, then passes that HTML to these parsers. For example:

```
Download upcoming-auctions HTML
  → parseAuctionListing()
  → collection.js saves each auction through database.js
```

The same pattern repeats for auction pages, catalogue pages and lots. **`drouot.js` returns extracted data; `collection.js` decides what to fetch next and when to save it.**

The parser can only use links and information present in the downloaded HTML. If Drouot changes its page structure or loads catalogue pages only through JavaScript, these parsing rules will need updating. It does not translate missing English text or use AI.

Collection

[collection.js](/Users/dcfinlay/Development/AI/EventsMonitor/src/collection.js) is the **job controller**. It decides what to collect next. It asks `drouot.js` to fetch and parse pages, asks `database.js` to save results, and asks `csv.js` to write exports. You do not start this file separately when using `npm start`.

Its call order is:

```
None
workerLoop()
  → processRun()
      → discover()
      → collectAuction() for each auction
          → collectLot() for each lot
      → exportCsvFiles()
```

What each function does

1. **[`workerLoop()` (line 172)](/Users/dcfinlay/Development/AI/EventsMonitor/src/collection.js:172) waits for work.** It asks PostgreSQL * for a queued job or an interrupted job whose lease has expired. When there is no job, it waits five seconds before checking again. This loop runs in the background while the API accepts requests.
2. **[`processRun()` (line 142)](/Users/dcfinlay/Development/AI/EventsMonitor/src/collection.js:142) manages one job.** It starts a heartbeat so the database knows the worker is still alive. It checks what this job has already done, discovers auctions if necessary, processes unfinished auctions, refreshes progress counts, writes both CSV files, and sets the final status to `completed` or `partial`. An unexpected error sets the job to `failed`.
3. **[`discover()` (line 19)](/Users/dcfinlay/Development/AI/EventsMonitor/src/collection.js:19) finds auctions.** It fetches the upcoming-auctions listing, passes the HTML to `drouot.js`, removes duplicate IDs, and immediately saves each auction's ID and URLs. It follows listing page numbers until it reaches the requested `maxAuctions` or the configured page limit. Finding fewer auctions than requested makes the run partial.
4. **[`collectAuction()` (line 67)](/Users/dcfinlay/Development/AI/EventsMonitor/src/collection.js:67) handles one auction.** It fetches its French and optional English pages and saves their details. If `includeLots` is true, it works through catalogue pages, saving each page's lot links. It reuses the French auction page as catalogue page 1, so that page is not downloaded twice. It then gets unsaved lots from PostgreSQL in batches of 20.

5. **[`collectLot()` (line 49)](/Users/dcfinlay/Development/AI/EventsMonitor/src/collection.js:49) handles one lot.** It fetches the French page and, if requested, the English page. It combines the extracted fields and saves the lot immediately. Missing required source text gives the lot a `partial` status; a fetch or parsing error gives it `failed`. The auction continues rather than discarding lots already saved.

How it resumes

A restarted job does not begin from an empty slate. [`processRun()` (line 148)](/Users/dcfinlay/Development/AI/EventsMonitor/src/collection.js:148) skips auctions already completed for that run. [`collectAuction()` (line 93)](/Users/dcfinlay/Development/AI/EventsMonitor/src/collection.js:93) skips catalogue pages marked complete. The database returns lots that still need details, so previously completed lots for that run are skipped. **PostgreSQL records each step as it happens**; the collector does not wait until the end to save everything.

Auctions and catalogue pages are processed in order. Within a lot batch, up to **two lots** are processed at once. The HTTP client in `drouot.js` still spaces out request starts.

The split between the files is this: **`drouot.js` answers "what does this page say?"; `collection.js` answers "which page do we fetch next, and what do we save now?"**

Postgres ToDO List *

PostgreSQL is acting as a **durable to-do list** for collection jobs. It does not download pages; the Node worker does that. PostgreSQL stores which job was requested, its progress, and whether a worker is currently handling it.

Here is what happens:

1. **You start a collection.** `POST /api/collections` inserts a row in [`drouot_collector.collection_runs` (line 4)](/Users/dcfinlay/Development/AI/EventsMonitor/migrations/001_collector.sql:4) with the options you supplied and `status = 'queued'`. The API [returns the row's ID immediately (line 86)](/Users/dcfinlay/Development/AI/EventsMonitor/src/server.js:86).
2. **The worker looks for a job.** [`workerLoop()` (line 172)](/Users/dcfinlay/Development/AI/EventsMonitor/src/collection.js:172) calls [`claimRun()` (line 50)](/Users/dcfinlay/Development/AI/EventsMonitor/src/database.js:50). That query finds one queued row, marks it `running`, and sets `lease_until` to 90 seconds in the future. A short database row lock prevents two workers from claiming the same job at once. If there is no job, the worker sleeps for five seconds.
3. **While collecting, it renews the lease.** Every 30 seconds, [`processRun()` (line 142)](/Users/dcfinlay/Development/AI/EventsMonitor/src/collection.js:142) calls [`heartbeat()` (line

68)](/Users/dcfinlay/Development/AI/EventsMonitor/src/database.js:68), moving `lease_until` another 90 seconds ahead. This means “the worker is still alive.”

4. **It records progress as it goes.** Auctions, catalogue pages and lots are saved to their tables. When the job finishes, [`finishRun()` (line 73)](/Users/dcfinlay/Development/AI/EventsMonitor/src/database.js:73) records `completed`, `partial` or `failed`.

The reason for the lease is a crash. If the worker disappears without updating its job, the `running` row remains in PostgreSQL, but its lease stops being renewed. After it expires, a restarted worker can claim that row and use the saved page and lot progress to continue. A graceful stop that records `failed` instead requires the [resume request (line 99)](/Users/dcfinlay/Development/AI/EventsMonitor/src/server.js:99).

So the five-second check is simply the Node worker asking, **“Is there a job for me?”** PostgreSQL is used because the job and its progress must survive an app restart. If you only wanted a manually run, one-off script with no job ID or recovery, this queue would be unnecessary; it supports the background and resume requirements in the project brief.

A collection job is **one row** in `drouot_collector.collection_runs`. It stores the requested options (such as how many auctions to collect and whether to include lots), the job’s status, start and finish times, a lease for crash recovery, progress counts, and the last error. It does **not** contain a list of downloaded auctions or lots. See the [table definition (line 4)](/Users/dcfinlay/Development/AI/EventsMonitor/migrations/001_collector.sql:4).

PostgreSQL tracks the individual work in separate rows:

Record	What it tells the app
`run_auctions` (line 66)	Which auctions belong to this run, and whether each is pending, completed, partial, or failed.
`catalogue_pages` (line 76)	Which catalogue pages were processed for this run. “Completed” means the page’s **lot links were saved**; it does not mean every lot’s details were downloaded.
`lots` (line 42)	Each lot’s details, `details_status`, and `last_collected_run_id`.

On resuming a run, the code skips an auction marked completed, skips catalogue pages marked completed, and downloads lot details only where the lot is **not completed in that same run**. A lot row merely existing is therefore not proof that its details were processed. See the [resume loop (line 142)](/Users/dcfinlay/Development/AI/EventsMonitor/src/collection.js:142), [page check (line 134)](/Users/dcfinlay/Development/AI/EventsMonitor/src/database.js:134), and [lot selection query (line 168)](/Users/dcfinlay/Development/AI/EventsMonitor/src/database.js:168).

A **new** run can download an already completed lot again to refresh it; the skip rule applies to completed lots in the *current* run. The counts on the job are progress summaries, while those individual records determine what gets skipped.